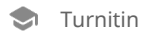


Author

server.c.pdf



Document Details

15 Pages

1,863 Words

10,177 Characters

File Name

unknown_filename

File Size

111.3 KB

36% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **61 Not Cited or Quoted 35%**
Matches with neither in-text citation nor quotation marks
-  **1 Missing Quotations 0%**
Matches that are still very similar to source material
-  **1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 20%  Internet sources
- 11%  Publications
- 35%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 61 Not Cited or Quoted 35%**
Matches with neither in-text citation nor quotation marks
- 1 Missing Quotations 0%**
Matches that are still very similar to source material
- 1 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 20% Internet sources
- 11% Publications
- 35% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works		
	Blue Mountain Hotel School on 2025-11-16		2%
2	Submitted works		
	CSU, San Jose State University on 2010-10-14		2%
3	Submitted works		
	Technological University Dublin on 2023-07-24		2%
4	Submitted works		
	University of Warwick on 2025-02-24		2%
5	Submitted works		
	Georgia Institute of Technology Main Campus on 2025-03-27		2%
6	Internet		
	codesearch.isocpp.org		2%
7	Submitted works		
	Hadassah Academic College on 2025-03-05		2%
8	Submitted works		
	University of Patras on 2024-01-22		1%
9	Internet		
	cpp.hotexamples.com		1%
10	Submitted works		
	AlHussein Technical University on 2026-01-20		1%

11	Internet	artcprogramming.blogspot.com	1%
12	Submitted works	Kingston University on 2018-03-06	1%
13	Internet	reviews.freebsd.org	1%
14	Submitted works	AlHussein Technical University on 2025-09-11	<1%
15	Internet	git.trinitydesktop.org	<1%
16	Submitted works	Oxford Brookes University on 2025-05-08	<1%
17	Submitted works	Silliman University on 2026-01-20	<1%
18	Internet	www.pudn.com	<1%
19	Submitted works	University of Northumbria at Newcastle on 2025-01-16	<1%
20	Internet	lists.openwall.net	<1%
21	Submitted works	AlHussein Technical University on 2025-09-11	<1%
22	Submitted works	AlHussein Technical University on 2026-01-25	<1%
23	Submitted works	Oxford Brookes University on 2020-04-24	<1%
24	Submitted works	AlHussein Technical University on 2025-09-13	<1%

25	Publication	Stevens, Thomas Fitzgerald. "A Lidar Based Semi-autonomous Collision Avoidanc...	<1%
26	Submitted works	University of Portsmouth on 2025-08-25	<1%
27	Internet	github.com	<1%
28	Internet	git.uibk.ac.at	<1%
29	Internet	gitea.iitdh.ac.in	<1%
30	Internet	www.justsoftwaresolutions.co.uk	<1%
31	Submitted works	(school name not available) on 2006-09-01	<1%
32	Submitted works	Oklahoma State University on 2025-02-14	<1%
33	Submitted works	Oklahoma State University on 2025-02-17	<1%
34	Internet	lists.fusionforge.zih.tu-dresden.de	<1%
35	Internet	www.coursehero.com	<1%
36	Internet	www.cs.cmu.edu	<1%
37	Submitted works	AlHussein Technical University on 2026-01-25	<1%
38	Submitted works	Georgia Institute of Technology Main Campus on 2025-11-05	<1%

39	Submitted works	Technological University Dublin on 2025-03-15	<1%
40	Submitted works	CSU, San Jose State University on 2009-05-14	<1%
41	Submitted works	UC, Boulder on 2017-11-06	<1%
42	Submitted works	UC, Boulder on 2017-12-08	<1%
43	Submitted works	University of Sydney on 2025-11-04	<1%
44	Submitted works	University of Warwick on 2026-01-19	<1%

// OS Assignment - dice game - server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <signal.h>
#include <time.h>
#include <errno.h>
```

// main setting

```
#define MP 3
#define WC 20
#define FIFO_PREFIX "/tmp/player_"
#define gamel "game.log"
#define srocesf "scores.txt"
```

```
typedef struct LogNode
{
    char message[256];
    struct LogNode *next;
} LogNode;
```

```
typedef struct
{
    LogNode *head;
    LogNode *tail;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
} LogQueue;
```

LogQueue log_queue;

```
typedef struct
{
    int PP[MP];
    int CT;
```

```

int game_active;
int CP;
int player_active[MP];
int FW;
int round;
char PN[MP][50];
int winner[MP];
pthread_mutex_t shm_lock;
pthread_mutex_t table_sync;
} GameInfo;

GameInfo *g_ptr = NULL;
int shm_fd;
pthread_t log_t, sch_t; // log_t = logger thread, sch_t = scheduler thread
volatile sig_atomic_t sr = 1; // sr = server running

// pre-declaration of functions
void ssm();
void csm(); // csm = cleanup shared memory
void *ltf(void *arg); // ltf = logger thread function
void *stf(void *arg); // stf = scheduler thread function
void hc(int player_id); // hc = handle client
void sigchld_handler(int sig);
void sigint_handler(int sig);
void log_message(const char *message);
void init();
void reset();
void ls(); // ls = load scores
void ss(); // ss = save scores

void ls() {
    FILE *f = fopen(srocesf, "r");
    if (f)
    {
        printf("[Server] Loading scores from %s...\n", srocesf);

        char line[256];
        int slot = 0;

        while (fgets(line, sizeof(line), f))
        {

```

```

        if (line[0] != '|') continue;
        if (strstr(line, "Dice Game")) continue;
        if (strstr(line, "Client Name")) continue;

        int score = 0;
        char *last_pipe = strrchr(line, '|');
        if (last_pipe)
        {
            *last_pipe = '\0';

            char *middle_pipe = strrchr(line, '|');
            if (middle_pipe)
            {
                score = atoi(middle_pipe + 1);

                if (slot < MP)
                {
                    g_ptr->winner[slot] = score;
                    printf(" - Slot %d Wins: %d (Loaded)\n", slot + 1, score);
                    slot++;
                }
            }
        }
        fclose(f);
    } else
    {
        printf("[Server] No previous scores found. Refreshing.\n");
        for (int i = 0; i < MP; i++)
        {
            g_ptr->winner[i] = 0;
        }
    }
}

void ss()
{
    FILE *f = fopen(srocesf, "w");
    if (f) {
        fprintf(f, "=====\n");
    }
}

```

12

```
fprintf(f, "|      Dice Game Results      |\n");
fprintf(f, "=====\n");
fprintf(f, "| %-20s | %-10s |\n", "Client Name", "Score");
fprintf(f, "=====\n");

for (int i = 0; i < MP; i++)
{
    char display_name[50];

    if (strlen(g_ptr->PN[i]) > 0)
    {
        strncpy(display_name, g_ptr->PN[i], 49);
        display_name[49] = '\0';
    } else
    {
        snprintf(display_name, sizeof(display_name), "Slot %d (Empty)", i + 1);
    }

    fprintf(f, "| %-20s | %-10d |\n", display_name, g_ptr->winner[i]);
}
```

26

```
fprintf(f, "=====\n");
fclose(f);
printf("[Server] Scores saved to %s (Table Format)\n", srocesf);
} else
{
    perror("Failed to save scores");
}
}
```

```
int main()
{
    printf("=== Dice Race Game Server - Started ===\n");
    printf("=== Server PID: %-24d ===\n", getpid());
    printf("Waiting for %d players to connect...\n\n", MP);
```

36

```
signal(SIGCHLD, sigchld_handler);
signal(SIGINT, sigint_handler);

ssm();
init();
```

```
pthread_mutexattr_t attr;
pthread_mutexattr_init(&attr);
pthread_mutexattr_setpshared(&attr, PTHREAD_PROCESS_SHARED);
pthread_mutex_init(&g_ptr->shm_lock, &attr);
pthread_mutex_init(&g_ptr->table_sync, &attr);
pthread_mutexattr_destroy(&attr);
```

```
log_queue.head = NULL;
log_queue.tail = NULL;
pthread_mutex_init(&log_queue.mutex, NULL);
pthread_cond_init(&log_queue.cond, NULL);
```

```
ls();
```

```
printf("[Main] Creating logger thread...\n");
int ret = pthread_create(&log_t, NULL, ltf, NULL);
```

```
if (ret != 0) {
    fprintf(stderr, "Failed to create logger thread: %s\n", strerror(ret));
    csm();
    exit(EXIT_FAILURE);
}
```

```
printf("[Main] Creating scheduler thread...\n");
ret = pthread_create(&sch_t, NULL, stf, NULL);
if (ret != 0) {
    fprintf(stderr, "Failed to create scheduler thread: %s\n", strerror(ret));
    csm();
    exit(EXIT_FAILURE);
}
```

```
printf("\n[Main] Both threads created successfully!\n");
printf("[Main] Thread count: 2 (logger + scheduler)\n\n");
```

```
log_message("Server started - waiting for players");
```

```
while (sr && g_ptr->CP < MP)
{
    for (int i = 0; i < MP; i++)
    {
        if (!g_ptr->player_active[i])
```

```

{
    char fifo_name[256];
    snprintf(fifo_name, sizeof(fifo_name), "%s%d_to_server", FIFO_PREFIX, i);

    if (access(fifo_name, F_OK) == 0)
    {
        pthread_mutex_lock(&g_ptr->shm_lock);
        g_ptr->player_active[i] = 1;
        g_ptr->CP++;
        pthread_mutex_unlock(&g_ptr->shm_lock);

        printf("Player %d connected (%d/%d)\n", i + 1,
            g_ptr->CP, MP);

        char log_msg[256];
        snprintf(log_msg, sizeof(log_msg), "Player %d connected", i + 1);
        log_message(log_msg);

        pid_t pid = fork();
        if (pid == 0)
        {
            hc(i);
            exit(0);
        } else if (pid < 0)
        {
            fprintf(stderr, "Fork failed for player %d: %s\n", i, strerror(errno));
            pthread_mutex_lock(&g_ptr->shm_lock);
            g_ptr->player_active[i] = 0;
            g_ptr->CP--;
            pthread_mutex_unlock(&g_ptr->shm_lock);
        }
    }
}

sleep(1);
}

if (g_ptr->CP == MP)
{
    printf("=== All players connected! Starting game ===\n");
}

```

```
pthread_mutex_lock(&g_ptr->shm_lock);
g_ptr->game_active = 1;
pthread_mutex_unlock(&g_ptr->shm_lock);
```

```
log_message("Game started - all players connected");
```

```
printf("[Main] Active processes:\n");
printf(" - Main server process (PID: %d)\n", getpid());
printf(" - Logger thread (TID: %lu)\n", (unsigned long)log_t);
printf(" - Scheduler thread (TID: %lu)\n", (unsigned long)sch_t);
printf(" - 3 child processes (forked for each player)\n");
printf("[Main] Total: 1 process + 2 threads + 3 child processes\n\n");
}
```

```
printf("[Main] Waiting for game to complete...\n");
while (sr && g_ptr->game_active)
{
    sleep(1);
}
```

```
printf("\n[Main] Game ended. Joining threads...\n");
pthread_mutex_lock(&log_queue.mutex);
pthread_cond_signal(&log_queue.cond);
pthread_mutex_unlock(&log_queue.mutex);
```

```
pthread_join(log_t, NULL);
printf("[Main] Logger thread joined\n");
pthread_join(sch_t, NULL);
printf("[Main] Scheduler thread joined\n");
```

```
printf("[Main] Cleaning up...\n");
csm();
```

```
printf("=== Server shutdown complete ===\n");
return 0;
}
```

```
void ssm()
{
```

```
shm_fd = shm_open("/dice_game_shm", O_CREAT | O_RDWR, 0666);
if (shm_fd == -1)
```

```

    {
        perror("shm_open failed");
        exit(EXIT_FAILURE);
    }

    if (ftruncate(shm_fd, sizeof(GameInfo)) == -1)
    {
        perror("ftruncate failed");
        close(shm_fd);
        shm_unlink("/dice_game_shm");
        exit(EXIT_FAILURE);
    }

    g_ptr = mmap(NULL, sizeof(GameInfo), PROT_READ | PROT_WRITE,
        MAP_SHARED, shm_fd, 0);
    if (g_ptr == MAP_FAILED)
    {
        perror("mmap failed");
        close(shm_fd);
        shm_unlink("/dice_game_shm");
        exit(EXIT_FAILURE);
    }
}

void csm()
{
    if (g_ptr != NULL)
    {
        pthread_mutex_destroy(&g_ptr->shm_lock);
        pthread_mutex_destroy(&g_ptr->table_sync);
        munmap(g_ptr, sizeof(GameInfo));
    }

    close(shm_fd);
    shm_unlink("/dice_game_shm");

    for (int i = 0; i < MP; i++)
    {
        char fifo_name[256];
        snprintf(fifo_name, sizeof(fifo_name), "%s%d_to_server", FIFO_PREFIX, i);
        unlink(fifo_name);
        snprintf(fifo_name, sizeof(fifo_name), "%s%d_from_server", FIFO_PREFIX, i);
    }
}

```

```

        unlink(fifo_name);
    }

```

1

```

        pthread_mutex_destroy(&log_queue.mutex);
        pthread_cond_destroy(&log_queue.cond);
    }

```

```

void init()
{
    memset(g_ptr, 0, sizeof(GameInfo));
    g_ptr->CT = 0;
    g_ptr->game_active = 0;
    g_ptr->CP = 0;
    g_ptr->FW = -1;
    g_ptr->round = 1;

```

```

    for (int i = 0; i < MP; i++)
    {
        g_ptr->PP[i] = 0;
        g_ptr->player_active[i] = 0;
        memset(g_ptr->PN[i], 0, 50);
    }
}

```

```

void reset()
{
    pthread_mutex_lock(&g_ptr->shm_lock);
    g_ptr->CT = 0;
    g_ptr->FW = -1;
    g_ptr->round = 1;
    for (int i = 0; i < MP; i++)
    {
        g_ptr->PP[i] = 0;
    }
    pthread_mutex_unlock(&g_ptr->shm_lock);
}

```

```

void log_message(const char *message)
{
    LogNode *new_node = (LogNode *)malloc(sizeof(LogNode));
    if (!new_node)

```

```
{
    perror("Failed to allocate log node");
    return;
}
```

18

```
time_t now = time(NULL);
char *time_str = ctime(&now);
time_str[strlen(time_str) - 1] = '\0';
snprintf(new_node->message, sizeof(new_node->message), "[%s] %s", time_str,
message);
new_node->next = NULL;
```

```
pthread_mutex_lock(&log_queue.mutex);
```

3

```
if (log_queue.tail == NULL)
{
    log_queue.head = new_node;
    log_queue.tail = new_node;
} else
{
    log_queue.tail->next = new_node;
    log_queue.tail = new_node;
}
```

25

```
pthread_cond_signal(&log_queue.cond);
pthread_mutex_unlock(&log_queue.mutex);
}
```

```
void *lrf(void *arg)
{
    printf("[Logger Thread] Started (TID: %lu)\n", (unsigned long)pthread_self());
    FILE *log_file;
```

```
while (sr || log_queue.head != NULL)
{
```

```
    pthread_mutex_lock(&log_queue.mutex);
```

```
    while (log_queue.head == NULL && sr)
    {
```

```
        pthread_cond_wait(&log_queue.cond, &log_queue.mutex);
```

```
    }
```

```
if(log_queue.head == NULL && !sr)
{
    pthread_mutex_unlock(&log_queue.mutex);
    break;
}
```

```
LogNode *current = log_queue.head;
log_queue.head = current->next;
if(log_queue.head == NULL)
{
    log_queue.tail = NULL;
}
```

```
pthread_mutex_unlock(&log_queue.mutex);
```

```
log_file = fopen(gamel, "a");
if(log_file)
{
    fprintf(log_file, "%s\n", current->message);
    fclose(log_file);
}
free(current);
}
```

```
printf("[Logger Thread] Shutting down\n");
return NULL;
}
```

```
void *stf(void *arg)
{
```

```
    printf("[Scheduler Thread] Started (TID: %lu)\n", (unsigned long)pthread_self());
```

```
while(sr)
{
    if(g_ptr->game_active)
    {
        pthread_mutex_lock(&g_ptr->shm_lock);
```

```
        for(int i = 0; i < MP; i++)
        {
```

```
if (g_ptr->PP[i] >= WC)
```

```
{
```

```
    g_ptr->FW = i;
```

```
    g_ptr->game_active = 0;
```

```
    char log_msg[256];
```

```
    snprintf(log_msg, sizeof(log_msg), "Game Over! %s wins!",
```

```
            g_ptr->PN[i]);
```

```
    log_message(log_msg);
```

```
    printf("%s\n", log_msg);
```

```
    pthread_mutex_unlock(&g_ptr->shm_lock);
```

```
    printf("[Scheduler Thread] Game ended, shutting down\n");
```

```
    return NULL;
```

```
}
```

```
}
```

```
pthread_mutex_unlock(&g_ptr->shm_lock);
```

```
}
```

```
usleep(1000);
```

```
}
```

```
printf("[Scheduler Thread] Shutting down\n");
```

```
return NULL;
```

```
}
```

```
void hc(int player_id)
```

```
{
```

```
    char fifo_read[256], fifo_write[256];
```

```
    snprintf(fifo_read, sizeof(fifo_read), "%s%d_to_server", FIFO_PREFIX, player_id);
```

```
    snprintf(fifo_write, sizeof(fifo_write), "%s%d_from_server", FIFO_PREFIX, player_id);
```

```
    usleep(2000);
```

```
    printf("[Child Process] Handling %s (slot %d, PID: %d, PPID: %d)\n",
```

```
          g_ptr->PN[player_id], player_id + 1, getpid(), getppid());
```

```
    srand(time(NULL) ^ getpid());
```

```
    int fd_read = open(fifo_read, O_RDONLY);
```

```
    int fd_write = open(fifo_write, O_WRONLY);
```

```

char buffer[256];

while (g_ptr->game_active)
{
    if (g_ptr->CT != player_id)
    {
        usleep(1000);
        continue;
    }

    memset(buffer, 0, sizeof(buffer));
    if (read(fd_read, buffer, sizeof(buffer)) > 0)
    {
        if (strncmp(buffer, "ROLL", 4) == 0)
        {
            int dice_roll = (rand() % 6) + 1;

            pthread_mutex_lock(&g_ptr->shm_lock);

            g_ptr->PP[player_id] += dice_roll;

            if (g_ptr->PP[player_id] >= WC)
            {
                g_ptr->PP[player_id] = WC;

                g_ptr->FW = player_id;
                g_ptr->game_active = 0;

                g_ptr->winner[player_id]++;
                printf("Player %d won! Total wins: %d\n",
                    player_id + 1, g_ptr->winner[player_id]);
                ss();
            } else
            {
                int next_turn = (g_ptr->CT + 1) % MP;
                g_ptr->CT = next_turn;

                if (next_turn == 0)
                {
                    g_ptr->round++;
                }
            }
        }
    }
}

```

```

    }
}

pthread_mutex_unlock(&g_ptr->shm_lock);

sprintf(buffer, "ROLLED %d", dice_roll);
write(fd_write, buffer, strlen(buffer) + 1);

```

6

```

char log_msg[256];
snprintf(log_msg, sizeof(log_msg), "Player %d (%s) rolled %d",
        player_id + 1, g_ptr->PN[player_id], dice_roll);
log_message(log_msg);
}
}
}

```

```

close(fd_read);
close(fd_write);
}

```

13

```

void sigchld_handler(int sig)
{
    int saved_errno = errno;
    pid_t pid;
    int status;
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {}
    errno = saved_errno;
}

```

34

```

void sigint_handler(int sig)
{
    printf("\n\n[SIGINT Handler] Caught interrupt signal\n");
    printf("[SIGINT Handler] Shutting down server gracefully...\n");
    sr = 0;
}

```

1

```

pthread_mutex_lock(&log_queue.mutex);
pthread_cond_signal(&log_queue.cond);
pthread_mutex_unlock(&log_queue.mutex);

if (g_ptr)
{

```

 40

```
pthread_mutex_lock(&g_ptr->shm_lock);  
g_ptr->game_active = 0;  
}  
}
```